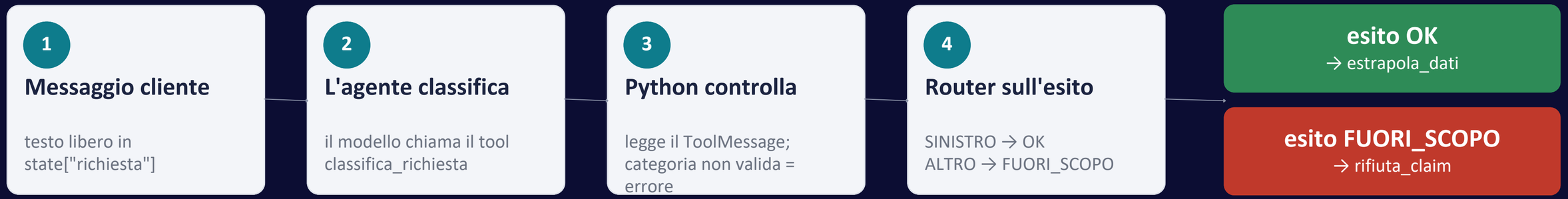
Progetto Agentic-AI

Assistente per la gestione di una pratica assicurativa

Andrea Tiani, Edoardo Spinelli, Antonio Lella, Francesco Favia
[13-10-2026]

Nodo 1 · orchestratore

Obiettivo: capire se il messaggio del cliente segnala un sinistro è fuori ambito, prima di avviare il resto del workflow.



L'orchestratore non risponde al cliente: classifica soltanto. La categoria è un campo del tool, non testo libero, e può essere solo SINISTRO o ALTRO.

Estratto (semplificato) da orchestratore()

```
@tool
def classifica_richiesta(motivo, categoria):
    categoria = categoria.strip().upper()
    if categoria not in ("SINISTRO", "ALTRO"):
        return json.dumps({"errore": "categoria non valida"})
    return json.dumps({"categoria": categoria, "motivo": motivo})

# ... l'agente chiama il tool, Python legge il ToolMessage
if risultato["categoria"] == "SINISTRO":
    return {"esito": "OK"}
return {"esito": "FUORI_SCOPO", "motivazione": risultato["motivo"]}
```

Tecnologie e metodi

- Agent ReAct (classe del professore) + tool calling
- Prompt con regole ed esempi per i casi limite
- Router LangGraph su OK | FUORI_SCOPO

Contributo al progetto

- Le richieste fuori scopo non attraversano il grafo
- Accetta anche domande senza codici
- Istruzioni date al sistema finiscono in ALTRO

Nodo 2 · estrapola_dati

Obiettivo: leggere dal messaggio del cliente i dati della pratica e metterli nello stato.

1 Messaggio
richiesta del cliente in testo libero

2 Agente LLM
legge il testo e chiama il tool

× **RuntimeError**

3 Tool
restituisce 6 campi in JSON

4 Pulizia
scarta i valori vuoti

5 Uniforma
codici maiuscoli, loss_type minuscolo

6 Campi richiesti?
5 su 6 sono obbligatori

× **errori: c mancante**

stato
dati

Se il messaggio non contiene un dato, resta vuoto: il modello non deve indovinare.

Estratto (semplificato) da estrapola_dati()

```
@tool
def registra_dati_richiesta(customer_id="", claim_id="", ...):
    return json.dumps({"customer_id": customer_id, ...})
grezzi = json.loads(m.content) # dal ToolMessage
dati = {k: v for k, v in grezzi.items() if v not in VUOTI}
dati[c] = dati[c].strip().upper() # codici
errori = [f"{c} mancante" for c in OBBLIGATORI if c not in dati]
return {**dati, "errori": errori}
```

Tecnologie e metodi

- Python + LangChain (@tool, ToolMessage)
- Agente LLM con regole nel system prompt
- Output strutturato via tool, in JSON

Contributo al progetto

- Trasforma il testo libero in dati dello stato
- Segnala i campi mancanti negli errori

Nodo 3 · verifica_polizza

Obiettivo: controllare che cliente e polizza esistano nei JSON, che la polizza sia del cliente, attiva e coerente con il sinistro.



Il primo controllo che fallisce interrompe tutto: esito RIFIUTATO con motivazione. I dati già letti restano nello stato.

Estratto (semplificato) da verifica_polizza()

```
cliente = CUSTOMERS.get(customer_id)
if not cliente:
    return rifiuta(f"Il cliente {customer_id} non esiste.")
# ... stesso schema per: polizza esiste?
if polizza["customer_id"] != customer_id:
    return rifiuta(f"La polizza ... non appartiene ...")
if polizza["coverage"] != loss_type:
    return rifiuta(f"La polizza ... copre ...")
return {**risultato, "esito": "OK"}
```

Tecnologie e metodi

- Python + file JSON (clienti, polizze)
- Dizionari indicizzati per ID
- Nodo LangGraph + router sull'esito

Contributo al progetto

- Scarta subito le pratiche non valide
- Salva nello stato nome, rischio, massimale

Nodo 4 · analizza_documenti

Obiettivo: controllare che il cliente abbia allegato tutti i documenti richiesti, entro MAX_RETRY tentativi.

1 Tutto presente?
richiesti già caricati

✓ OK

2 Tentativi esauriti?
retry ≥ MAX_RETRY

× TROPPI TENTATIVI

3 Pausa e richiesta
interrupt() al cliente

4 Documento valido?
tra i validi e non doppio

5 Contatore +1
ogni risposta è un tentativo

6 Nuovo controllo
mancanti() è vuoto?

↻ AMBIGUO: torna al 3

esito **OK**

Ogni risposta conta come un tentativo, anche se il documento non è valido.

Estratto (semplificato) da analizza_documenti()

```
risposta = interrupt({"mancanti": mancanti(), ...})
doc = str(risposta).strip().lower()
if doc in DOCUMENTI_VALIDI and doc not in presenti:
    presenti.append(doc)
retry += 1
if not mancanti(): return {..., "esito": "OK"}
if retry >= MAX_RETRY: return {..., "esito": "TROPPI_TENTATIVI"}
return {..., "esito": "AMBIGUO"}
```

Tecnologie e metodi

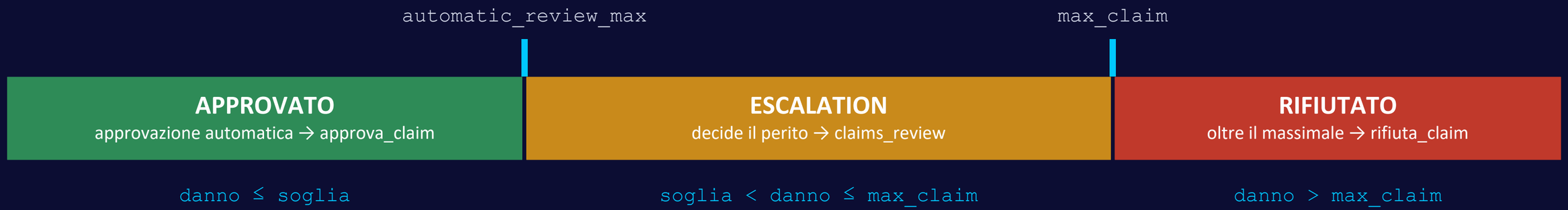
- Python + LangGraph (StateGraph, Command)
- interrupt() per il human-in-the-loop
- MemorySaver + thread_id, requirements.json

Contributo al progetto

- Raccoglie i documenti mancanti dal cliente
- Passa a verifica_budget con la soglia

Nodo 5 · verifica_budget

Obiettivo: confrontare il danno stimato con la soglia di approvazione automatica e con il massimale di polizza, e scegliere il percorso della pratica.



Si controlla prima il massimale: se il danno lo supera, il resto non conta. Soglia e massimale arrivano dallo stato, già compilato dai nodi a monte (analizza_documenti e verifica_polizza).

Estratto (semplificato) da verifica_budget()

```
danno     = float(state["estimated_damage"])
max_claim = float(state["max_claim"])
soglia    = float(state["automatic_review_max"])

# prima il massimale: se lo supera, il resto non conta
if danno > max_claim:
    return {"esito": "RIFIUTATO", "motivazione": ...}
if danno <= soglia:
    return {"esito": "APPROVATO", "motivazione": ...}
return {"esito": "ESCALATION", "motivazione": ...}
```

Tecnologie e metodi

- Confronto numerico in Python, senza LLM
- Legge solo dallo stato (danno, massimale, soglia)
- Router a tre vie sull'esito

Contributo al progetto

- Decisione economica deterministica e ripetibile
- Motivazione con gli importi, riusata a valle
- Al perito arrivano solo le pratiche intermedie

Nodo 6 · claims_review

Ruolo: scrive un report descrittivo della pratica per il perito. Non valuta e non decide.



Il testo generato è valido se riporta l'importo e il risk_tier e non contiene «consiglio» o «raccomando …».

Problema risolto

- Il perito ha bisogno di un riepilogo chiaro
- Il report resta fedele ai fatti dello stato
- Niente consigli né giudizi

Interazione con il sistema

- Attivo sul ramo ESCALATION
- Legge solo lo stato, scrive report_perito
- chiama_perito lo mostra al perito

Tecnologie e meccanismi

- LLM con SystemMessage + HumanMessage
- Calcoli fatti dal codice, non dal modello
- Validazione e report di riserva

Nodo 7 · chiama_perito

Obiettivo: sul ramo ESCALATION sottoporre la pratica al perito e raccogliere la sua decisione sì / no.

1 **Report pronto**

report_perito scritto da claims_review

2 **Pausa: interrupt()**

il grafo si ferma e invia il payload

3 **Il perito risponde**

si / no (risposta non valida: richiesta di nuovo)

esito

APPROVATO

RIFIUTATO

Il payload contiene: pratica, cliente, tipo di sinistro, danno stimato, soglia automatica, massimale, documenti e report.

Estratto da chiama_perito()

```
# la decisione del perito non va mai indovinata
while True:
    risposta = interrupt(payload)
    scelta = str(risposta).strip().lower()
    if scelta in RISPOSTE_SI:
        return {"esito": "APPROVATO", ...}
    if scelta in RISPOSTE_NO:
        return {"esito": "RIFIUTATO", ...}
    payload = {**payload, "messaggio": ... "non valida" ...}
```

Tecnologie e metodi

- LangGraph: interrupt() e Command(resume)
- Insiemi RISPOSTE_SI / RISPOSTE_NO
- Router su esito APPROVATO | RIFIUTATO

Contributo al progetto

- Decisione umana sulle pratiche oltre soglia
- Mostra al perito anche il report dell'agente

Nodo 8 · approva_claim

Obiettivo: chiudere la pratica come approvata e calcolare l'indennizzo da riconoscere al cliente (danno meno franchigia).

1 — Pratica approvata
arriva da verifica_budget o dalla decisione del perito

2 — Legge lo stato
estimated_damage e deductible

3 — Calcola l'indennizzo
max(0, danno − franchigia)

4 — Aggiorna lo stato
esito, indennizzo e motivazione

esito APPROVATO
→ formula_esito

Il nodo non decide nulla: l'approvazione è già stata presa a monte. Si limita a quantificare quanto spetta al cliente.

Estratto (semplificato) da approva_claim()

```
danno      = float(state["estimated_damage"])
franchigia = float(state["deductible"])
indennizzo = max(0.0, danno - franchigia)    # mai negativo

precedente = state.get("motivazione", "")
dettaglio  = f"Indennizzo riconosciuto: {indennizzo:.2f} € ..."

return {"esito": "APPROVATO",
        "indennizzo": indennizzo,
        "motivazione": f"{precedente} {dettaglio}".strip()}
```

Tecnologie e metodi

- Python puro: calcolo deterministico, nessun LLM
- Nodo raggiunto da due rami (automatico e perito)
- max(0, …) evita indennizzi negativi

Contributo al progetto

- Un solo punto in cui si calcola l'importo pagato
- Mantiene la motivazione del nodo precedente
- Passa a formula_esito un indennizzo già pronto

Nodo 9 · rifiuta_claim

Obiettivo: chiudere come non accolta ogni pratica che esce dal flusso, mantenendo l'esito e la motivazione dei nodi precedenti.

1 — Quattro rami

da orchestratore, polizza, documenti, budget e perito

2 — Legge lo stato

esito e motivazione già scritti a monte

3 — Sceglie il messaggio

un testo per ogni tipo di esito

4 — Azzera l'indennizzo

indennizzo = 0.0

- **FUORI_SCOPO** → formula_esito
- **TROPPI_TENTATIVI** → formula_esito
- **RIFIUTATO** → formula_esito

Un solo nodo raccoglie tutti i rifiuti: ogni esito mantiene la sua etichetta, così formula_esito può rispondere al cliente nel modo giusto.

Estratto (semplificato) da rifiuta_claim()

```
esito  = state.get("esito")
motivo = state.get("motivazione", "").strip()
if esito == "FUORI_SCOPO":
    motivazione = "Richiesta fuori ambito ..." + motivo
elif esito == "TROPPI_TENTATIVI":
    motivazione = motivo or "Documenti incompleti ..."
else:
    esito, motivazione = "RIFIUTATO", motivo or "..."

return {"esito": esito, "indennizzo": 0.0,
        "motivazione": motivazione}
```

Tecnologie e metodi

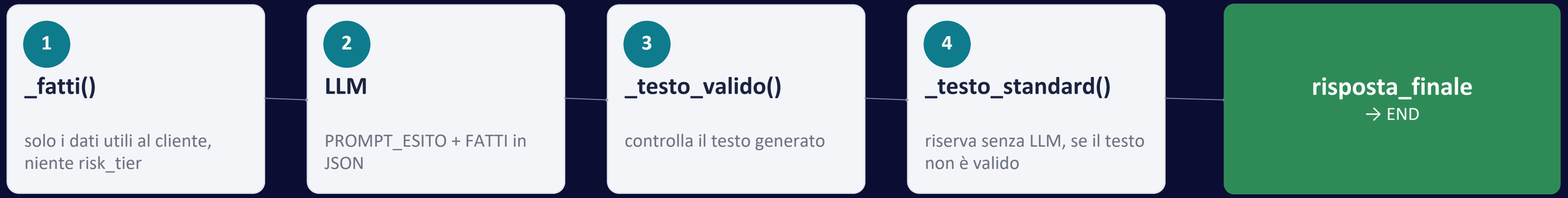
- Python puro: if/elif sull'esito ricevuto
- Stato condiviso di LangGraph come unico canale
- Motivazione di riserva se manca

Contributo al progetto

- Nodo di chiusura comune a quattro rami del grafo
- Nessuna pratica rifiutata resta senza motivazione
- Indennizzo sempre a zero nei rifiuti

Nodo 10 · formula_esito

Obiettivo: trasformare l'esito tecnico della pratica in un messaggio chiaro e cortese per il cliente.



Il testo del modello viene accettato solo se supera il controllo; altrimenti il cliente riceve comunque un messaggio corretto.

Estratto (semplificato) da formula_esito()

```
fatti = _fatti(state)
messaggi = [SystemMessage(content=PROMPT_ESITO),
            HumanMessage(content="FATTI:\n" + json.dumps(fatti))]
testo = llm.invoke(messaggi).content
testo = re.sub(r"<think>.*?</think>", "", testo,
               flags=re.DOTALL).strip()

if not _testo_valido(testo, fatti):
    testo = _testo_standard(fatti)

return {"risposta_finale": testo}
```

Tecnologie e metodi

- LLM con SystemMessage + HumanMessage
- Fatti e importi preparati dal codice, non dal modello
- Validazione della risposta e messaggio di riserva

Contributo al progetto

- Il cliente legge una risposta chiara e coerente con l'esito
- Nessun codice interno nel messaggio
- Il flusso non dipende solo dal modello

Grazie per l’attenzione